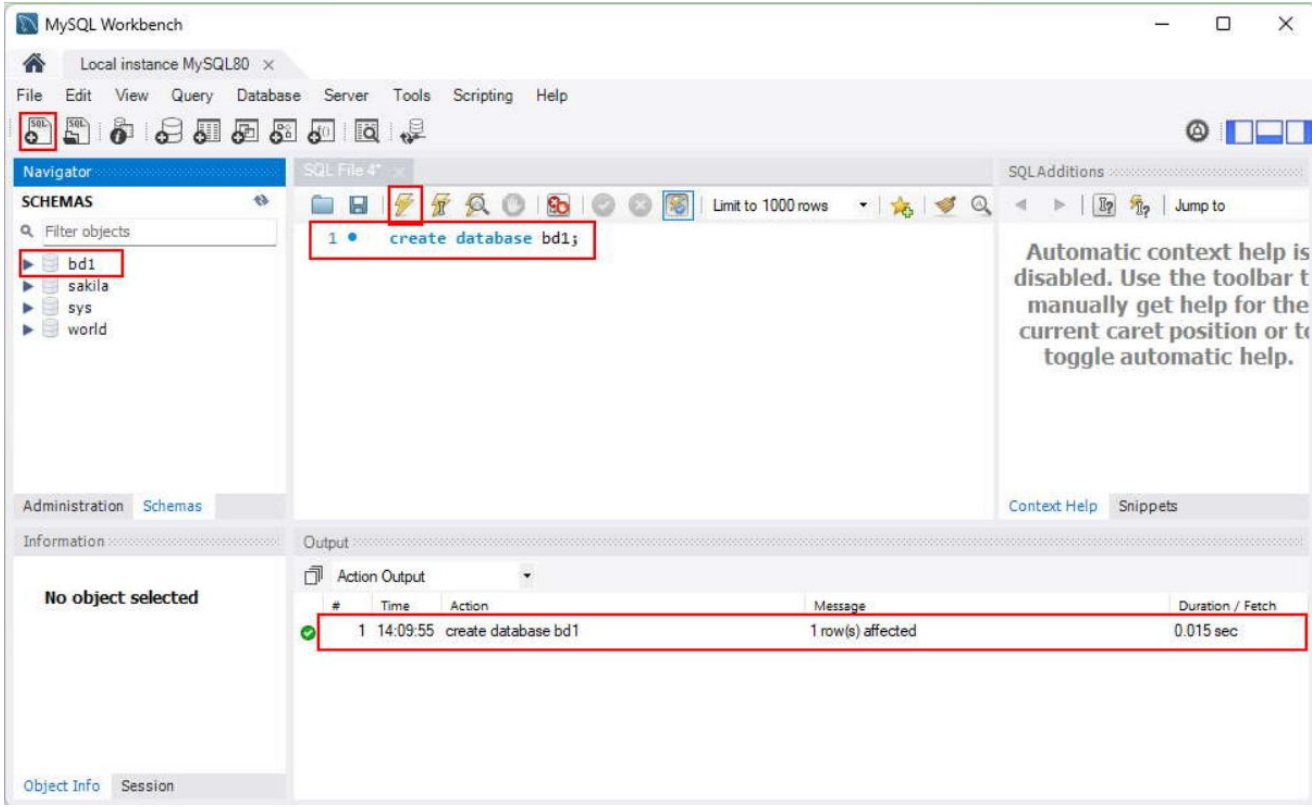
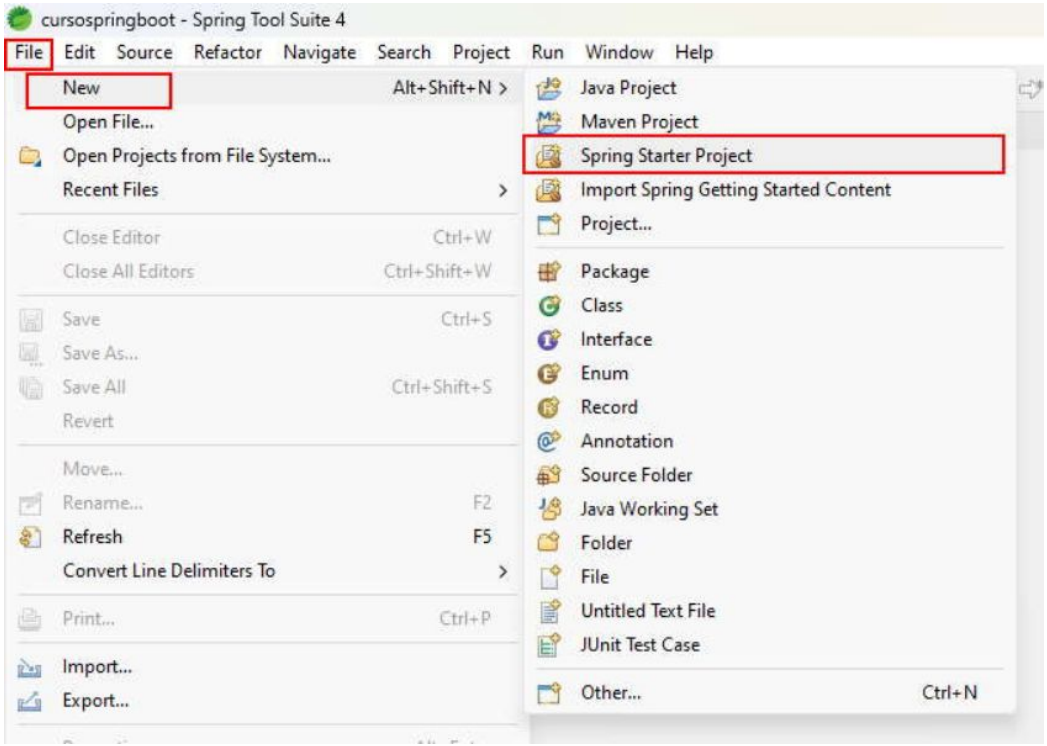
ACTIVIDAD

CREACIÓN DE BASE DE DATOS DESDE SPRING BOOT

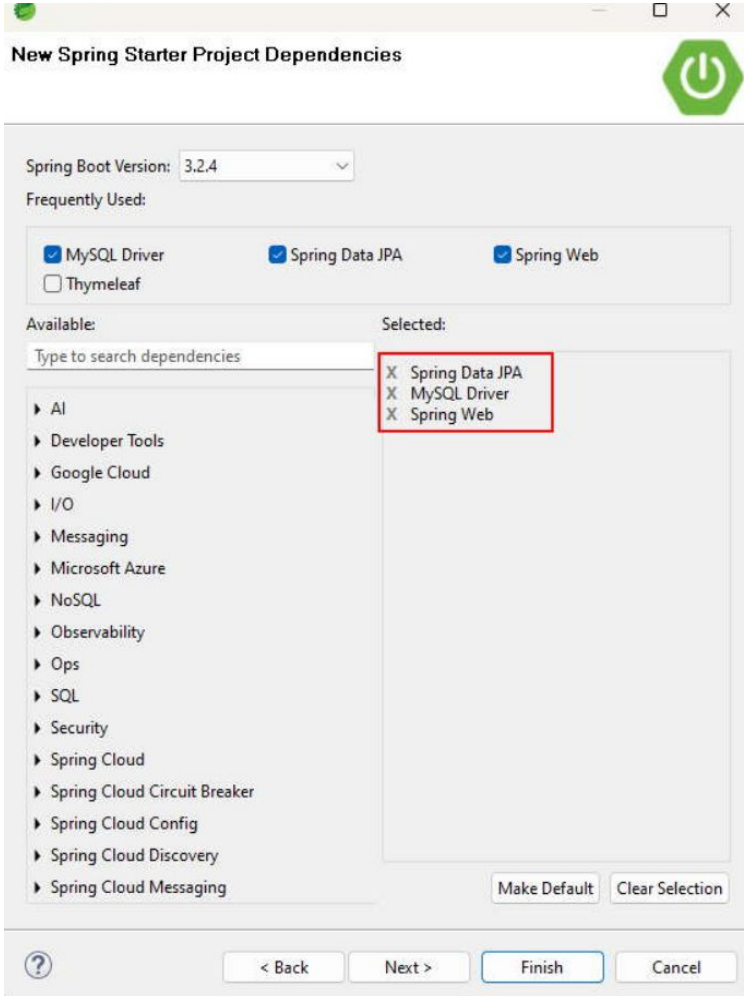
Crear una base de datos



Desde el menú de opciones seleccionamos File-> New -> Spring Starter Project:



En el segundo diálogo seleccionamos ahora tres dependencias



Ahora especificamos que accederemos a la base de datos 'bd1', dicha actividad se hace en el archivo 'application.properties':

```
spring.application.name=proyecto007
spring.datasource.url=jdbc:mysql://localhost:3306/bd1?useSSL=false
spring.datasource.username=root
spring.datasource.password=123456
spring.jpa.hibernate.ddl-auto=update
```

Procedemos a crear la carpeta 'model' y en la misma la clase 'Chiste'

```
package com.example.demo.model;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

@Entity
public class Chiste {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;
    private String texto;
    private String autor;

    public Chiste() {
    }

    public Chiste(int id, String texto, String autor) {
        this.id = id;
        this.texto = texto;
        this.autor = autor;
    }
```

```
    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getTexto() {
        return texto;
    }

    public void setTexto(String texto) {
        this.texto = texto;
    }

    public String getAutor() {
        return autor;
    }

    public void setAutor(String autor) {
        this.autor = autor;
    }
}
```

Ahora creamos la carpeta 'repository' y dentro creamos la interface 'ChisteRepository':

```
package com.example.demo.repository;

import org.springframework.data.jpa.repository.JpaRepository;

import com.example.demo.model.Chiste;

public interface ChisteRepository extends JpaRepository<Chiste, Integer> {

}
```

Creamos la carpeta 'service' y dentro de la misma la clase 'ChisteService'.

```
package com.example.demo.service;

import java.util.List;
import java.util.Random;

import org.springframework.stereotype.Service;

import com.example.demo.model.Chiste;
import com.example.demo.repository.ChisteRepository;

@Service
public class ChisteService {
    ChisteRepository repo;

    public ChisteService(ChisteRepository repo) {
        this.repo = repo;
    }

    public List<Chiste> retornarChistes() {
        return repo.findAll();
    }
```

```java
    public void agregarChiste(Chiste chiste) {
        repo.save(chiste);
    }

    public void eliminarChiste(int id) {
        repo.deleteById(id);
    }

    public void actualizarChiste(int id, Chiste chiste) {
        Chiste chisteModificado = repo.findById(id).get();
        chisteModificado.setTexto(chiste.getTexto());
        chisteModificado.setAutor(chiste.getAutor());
        repo.save(chisteModificado);
    }

    public Chiste obtenerChiste(int id) {
        return repo.findById(id).get();
    }

    public Chiste obtenerChisteAleatorio() {
        List<Chiste> todosLosChistes = repo.findAll();
        Random rand = new Random();
        int indiceAleatorio = rand.nextInt(todosLosChistes.size());
        return todosLosChistes.get(indiceAleatorio);
    }

}
```

Finalmente nos queda crear los endpoints en el controlador. Creamos la carpeta controller y dentro la clase 'ChisteController':

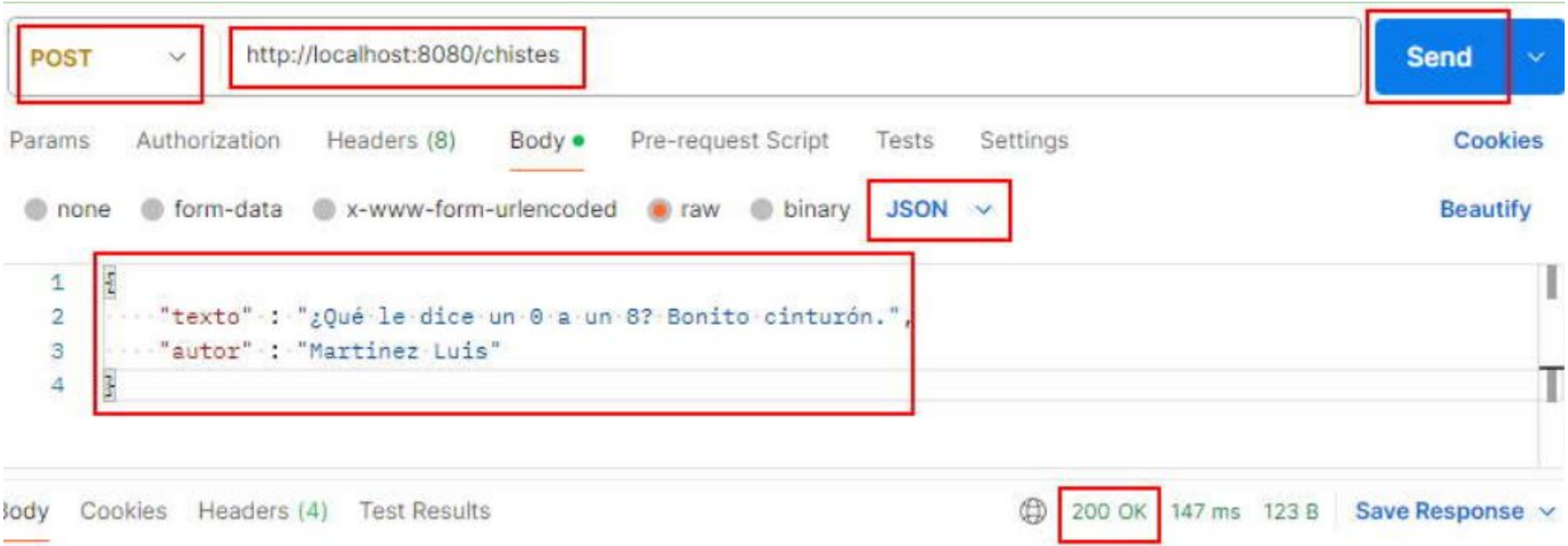
```
package com.example.demo.controller;

import java.util.List;

import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RestController;

import com.example.demo.model.Chiste;
import com.example.demo.service.ChisteService;

@RestController
public class ChisteController {
```

```java
private final ChisteService chisteService;

public ChisteController(ChisteService chisteService) {
    this.chisteService = chisteService;
}

// Endpoint para obtener todos los chistes
@GetMapping("/chistes")
public List<Chiste> obtenerTodosLosChistes() {
    return chisteService.retornarChistes();
}

// Endpoint para agregar un nuevo chiste
@PostMapping("/chistes")
public void agregarNuevoChiste(@RequestBody Chiste chiste) {
    chisteService.agregarChiste(chiste);
}

// Endpoint para eliminar un chiste por su ID
@DeleteMapping("/chistes/{id}")
public void eliminarChiste(@PathVariable int id) {
    chisteService.eliminarChiste(id);
}
```

```
    // Endpoint para actualizar un chiste existente
    @PutMapping("/chistes/{id}")
    public void actualizarChiste(@PathVariable int id, @RequestBody Chiste nuevoChiste) {
        chisteService.actualizarChiste(id, nuevoChiste);
    }

    // Endpoint para obtener un chiste por su ID
    @GetMapping("/chistes/{id}")
    public Chiste obtenerChistePorId(@PathVariable int id) {
        return chisteService.obtenerChiste(id);
    }

    // Endpoint para obtener un chiste aleatorio
    @GetMapping("/chistes/aleatorio")
    public Chiste obtenerChisteAleatorio() {
        return chisteService.obtenerChisteAleatorio();
    }

}
```

Ahora procedamos a probar los 'endpoints' desde Postman.

Agreguemos un chiste a nuestra tabla de la base de datos:



Luego de insertar otros chistes, probemos los otros endpoints.

Recuperar todos los chistes:

GET http://localhost:8080/chistes Send

Params Authorization Headers (6) Body Pre-request Script Tests Settings Cookies

Query Params

Key	Value	Bulk Edit

ody Cookies Headers (5) Test Results 200 OK 92 ms 443 B Save Response

Pretty Raw Preview Visualize JSON


```
1  [
2      {
3          "id": 1,
4          "texto": "¿Qué le dice un 0 a un 8? Bonito cinturón.",
5          "autor": "Martinez Luis"
6      },
7      {
8          "id": 2,
9          "texto": "¿Qué hace una abeja en el gimnasio? ¡Zum-ba!",
10         "autor": "Anonimo"
11     },
12     {
13         "id": 3,
14         "texto": "¿Cuál es el colmo de un electricista? Que su hijo sea un apagado",
15         "autor": "Anonimo"
16     }
17 ]
```

Recuperar un chiste por su código

GET http://localhost:8080/chistes/1 Send

Params Authorization Headers (6) Body Pre-request Script Tests Settings Cookies

Query Params

	Key	Value	Bulk Edit

Body Cookies Headers (5) Test Results 200 OK 13 ms 252 B Save Response

Pretty Raw Preview Visualize JSON


```
1 {
2     "id": 1,
3     "texto": "¿Qué le dice un 0 a un 8? Bonito cinturón.",
4     "autor": "Martinez Luis"
5 }
```

Borrar un chiste:

DELETE http://localhost:8080/chistes/1 Send

Params Authorization Headers (6) Body Pre-request Script Tests Settings Cookies

Query Params

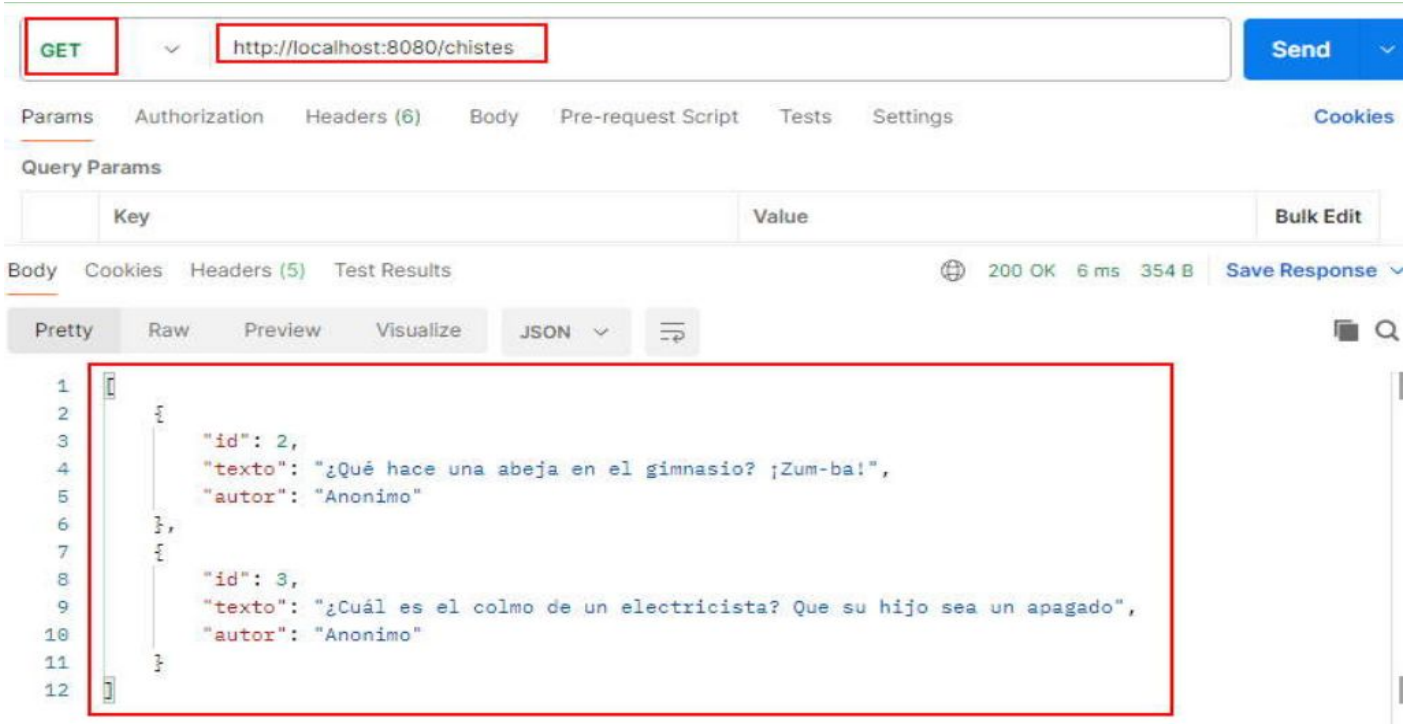
	Key	Value	Bulk Edit
	Key	Value	

Body Cookies Headers (4) Test Results 200 OK 13 ms 123 B Save Response

Pretty Raw Preview Visualize Text

1

Luego de borrarlo si consultamos todos los chistes, debería estar borrado el chiste con código 1:



Los últimos endpoints que podemos probar son el que modifica un registro de la tabla:

PUT http://localhost:8080/chistes/2 Send

Params Authorization Headers (8) Body Pre-request Script Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary JSON Beautify

```
1 {
2     "texto" : "¿Por qué estás hablando con la pared? ¡Porque la mesa no me responde!",
3     "autor" : "Lopez Pablo"
4
5 }
```

Body Cookies Headers (4) Test Results 200 OK 13 ms 123 B Save Response

Pretty Raw Preview Visualize Text

```
1
```

Y el que retorna un chiste al azar:

GET http://localhost:8080/chistes/aleatorio Send

Params Authorization Headers (6) Body Pre-request Script Tests Settings Cookies

Query Params

	Key	Value	Bulk Edit
	Key	Value	

Body Cookies Headers (5) Test Results 200 OK 5 ms 278 B Save Response

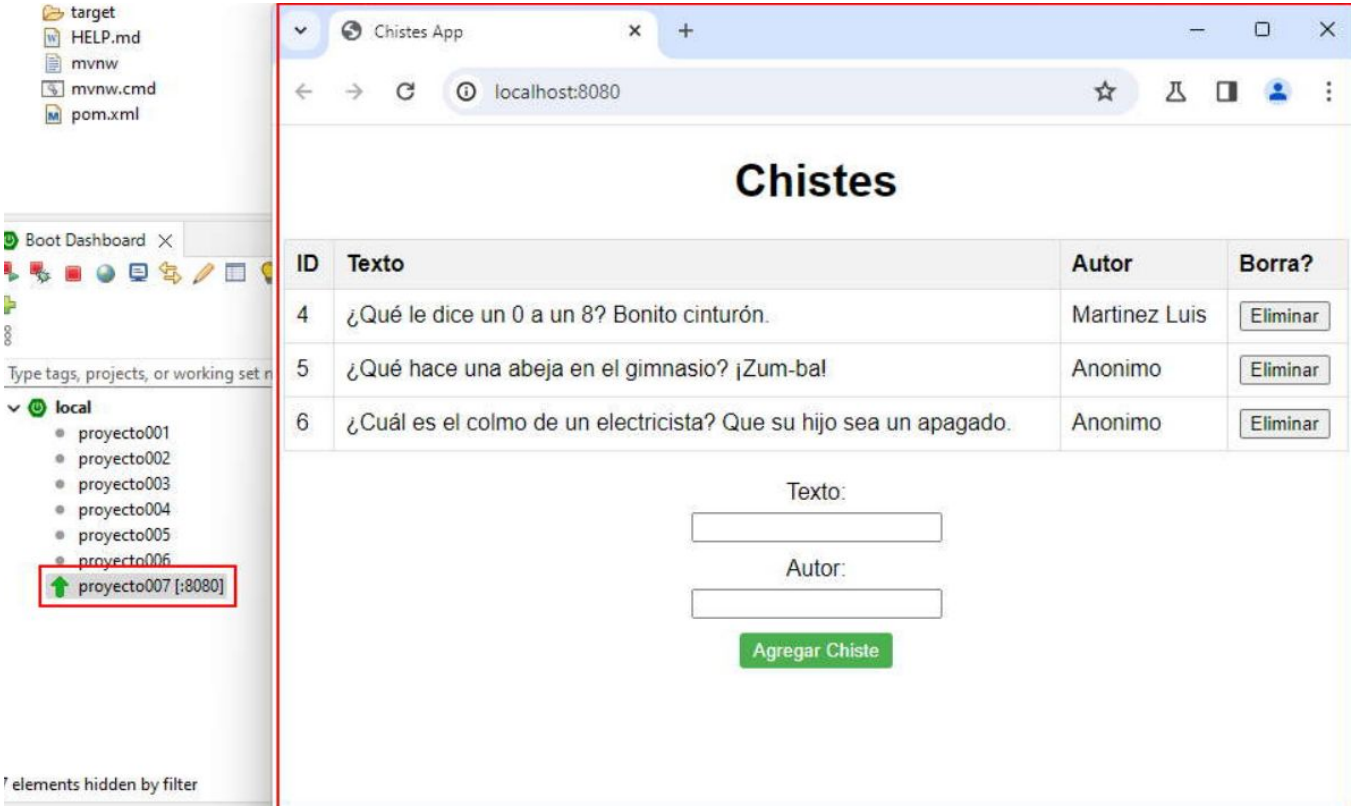
Pretty Raw Preview Visualize JSON


```
1 {
2     "id": 2,
3     "texto": "¿Por qué estás hablando con la pared? ¡Porque la mesa no me responde!",
4     "autor": "Lopez Pablo"
5 }
```

Accesos a los endpoints desde un sitio web.

Agreguemos los archivos index.html, index.js y estilos.css en el proyecto para consumir los endpoints desde una aplicación web.

La vista quedaria asi y se ingresará los datos a la bd



¡TERMINE!
POR FIN TERMINE
MI TAREA
memegenerator.es